

# 1 Plan: Clean Rapier-based sim-dev rewrite (v2-only)

## 1.1 Status

| Phase                                                                                              | State         | Notes                                                                                             |
|----------------------------------------------------------------------------------------------------|---------------|---------------------------------------------------------------------------------------------------|
| 1 — Project skeleton + scene                                                                       | ☐ done        |                                                                                                   |
| 2 — Robot loader (no physics)                                                                      | ☐ done        |                                                                                                   |
| 3 — Static physics + drop test                                                                     | ☐ done        |                                                                                                   |
| 4 — Map loading (JSON parity)                                                                      | ☐ done        | model loading async; trimesh colliders for <code>immovable</code> replaced                        |
| 5 — Custom raycast wheel force model                                                               | ☐ done        | <code>DynamicRayCastVehicleController</code> ; drift / turn fixes landed ( <code>e52aca9</code> ) |
| 5b — UI scaffolding (camera modes, splash, presets, text labels, remember-stage, runtime controls) | ☐ done        |                                                                                                   |
| 6 — Stage object collider visualization                                                            | ☐ done        | wireframes for floor / cube / cylinder / model                                                    |
| 7 — Debug menu (lil-gui)                                                                           | ☐ done        | all folders wired; most params mutate live (mass/radius require body recreation)                  |
| 8 — Ramps and behavior validation                                                                  | ☐ done        | holds on all ramps (5°–40°); no slip; descent controlled; edge drop clean                         |
| 9 — Replace <code>sim-dev/</code> with <code>sim-v2/</code>                                        | ☐ not started | needs <code>React.StrictMode</code> re-enable first                                               |

## 1.2 Context

The current `sim-dev/` drives the robot with a kinematic-ish approach: 2 sphere wheel colliders attached to the chassis body, velocities set directly via `setLinvel`/`setAngvel`, plus an uprightness gate and wheel raycast probes. This works on flat ground but **produces** wrong wheel positions, no real suspension, no slip, no holding-on-ramps behavior. The plan: replace it with a small **custom raycast wheel force model** — per-wheel downward raycast → suspension spring force + per-wheel longitudinal traction (engine/brake) + side-friction force, all applied through `RigidBody.applyImpulseAtPoint` on the chassis. (Earlier draft used Rapier’s `DynamicRayCastVehicleController`; that was dropped — see Phase 5 below.)

The current `sim-dev/` also depends on `front-end/src/components/js-simulator/` (the V1 OBJ loader, scene/camera/lights, stage JSON loader). The new tree will not. Once the new tree drives v2 cleanly on flat ground + ramps, it replaces `sim-dev/`.

The intended outcome is a small, owned, easy-to-tune simulator that gets wheel mechanics *right* — correct positions, real suspension, real friction — so the rest of the GSoC project (sensors, programs, multi-robot) sits on a trustworthy foundation.

## 1.3 Scope (v2 only, no V1, no js-simulator import)

- One robot: v2, loaded from `sim-v2/public/js-simulator/models/robots/v2/*.stl` so this tree owns the assets it serves.
- Stages: a flat plane stub (phases 1–3) replaced by JSON stages with parity to the V1 js-simulator schema (phase 4); ramp set added in phase 8.
- Two driving wheels (raycast) + one caster (passive free-roll sphere collider on chassis — see “Caster decision” below).
- Debug GUI via `lil-gui` (small, well-known, `dat.gui`-style folders/sliders/buttons).

## 1.4 New directory

`sim-v2/` at repo root, sibling to `sim-dev/`. Same Vite + React + TS stack so the eventual swap is mechanical. No imports from `front-end/src/components/js-simulator/`.

**Asset strategy:** STL parts for v2 are loaded from the existing public URL path (`/js-simulator/models/robots/v2/*.stl`) — they stay where they are, the new tree just fetches them. The wheel mesh (`wheel.obj` + `wheel.mtl`) is copied into `sim-v2/public/wheel/` so the new tree owns its own wheel asset and doesn't import from the V1 robot folder.

## 1.5 Caster decision (load-bearing, picked here so Phase 3 doesn't regress)

`DynamicRayCastVehicleController` is car-shaped: N steered/driven raycast wheels, no concept of a passive caster. Three options were considered:

- (A) 2 raycast wheels for drive + caster as a free `Ball` collider on the chassis. Caster contributes weight, contact, and stability the way the real robot's caster does. Selected.
- (B) 3 raycast wheels (caster as a third raycast wheel, non-driven/non-steered). Cheaper but loses the rolling-ball physics of a real caster and is a hack against the controller's intent.
- (C) Lift caster out of physics entirely (visual only). Wrong — the caster carries a third of the weight; ignoring it changes balance.

Going with (A). If (A) misbehaves on ramps (caster ball sticking, popping), fall back to (B).

## 1.6 Phase ordering (each phase ends with a verifiable checkpoint)

### 1.6.1 Phase 1 — Project skeleton + scene (no physics, no robot)

1. Scaffold `sim-v2/` (package.json, tsconfig, vite.config, index.html, `src/main.tsx`).
2. Bootstrap Three.js: renderer, scene, perspective camera, `OrbitControls`, ambient + directional light, ground plane (`PlaneGeometry`, 20×20m, gridhelper on top), RAF render loop.
3. Top-right **orientation gizmo**: a second small `WebGLRenderer` viewport (or a small scene rendered into a corner via `setViewport`) showing an `AxesHelper` whose rotation tracks the robot's chassis rotation. World axes can be a separate static helper for reference.
4. Window resize + DPR handling.

**Verify:** open the page, see grid + lighting + orbit camera, see two axis triads (world reference + corner gizmo) responding to the camera.

### 1.6.2 Phase 2 — Load v2 robot, correct orientation, no physics

1. STL loader (`STLLoader` from `three/examples/jsm/loaders/STLLoader`).
2. Reimplement the v2 part assembler. Mirror the structure of `sim-dev/src/robot/robotV2.ts` for known-good values:
  - Outer pivot `Group` with `rotation.x = - $\pi/2$`  (Fusion Z-up → world Y-up).
  - Inner parts at `PART_DEFAULTS` mm offsets, scaled by 0.001.
  - Right-fender mirror trick (`right_fender.stl` is broken — mirror left).
  - `front_base.rotation.z =  $\pi$`  (face forward).
  - Color map per part.
3. Wheels: load `caster.obj` once (caster) + a wheel STL/OBJ (TODO: confirm asset — see Q2 below). Place left/right wheels at fender-center mount points using the existing constants.
4. Caster mesh placed at its mount point.

**Verify:** robot stands on grid plane, body upright, wheels visually at the fender mounts, caster underneath at the right spot, orientation gizmo's axes match what the robot looks like (robot's "forward" matches gizmo's forward).

### 1.6.3 Phase 3 — Static physics + chassis drop test (no vehicle controller yet)

1. Init Rapier (@dimforge/rapier3d-compat): `await RAPIER.init()`, world with gravity `(0, -9.81, 0)`.

2. Static ground: cuboid collider at  $y=0$ .
3. Chassis rigid body (dynamic). For collision shape use the `v2 _coacd` convex hulls (compound `ConvexHull` colliders) — this asset set is already in the repo. Mass set explicitly (target  $\sim 1.5\text{--}2.5$  kg total — confirm in Q3).
4. Caster: Ball collider on the same body, radius from caster mesh bbox.
5. Mesh sync: each frame, copy `body.translation()` and `body.rotation()` to the visual pivot, applying the body-local visual offset rotated by the body quaternion (the fix from FINDINGS.md).
6. Stepping: `world.step()` at fixed dt ( $1/60$ ), accumulator pattern in the RAF loop.

**Verify:** spawn robot 0.5m above ground, watch it fall and rest stably. Body upright, caster contacting, no jitter, mesh stays glued to body.

#### 1.6.4 Phase 4 — Map loading (JSON stage parity)

Goal: load existing js-simulator stages without modification. Stage files are bundled from `sim-v2/src/stages/data/*.json` and referenced assets live under `sim-v2/public/js-simulator/`. The schema and asset URLs match V1, so the same JSON shape drives both simulators.

**Schema observed in the V1 stages** (no formal spec, derived from reading `stage_white_rect.json`, `stage_ramp.json`, `stage_object.json`):

- **floor** — dimensions: `[w, h]` (per-tile; texture tiles via `repeat: [u, v]`), `material.color` (string or decimal RGB), optional `texture` URL, optional `offset`.
- **cube** — dimensions: `[w, h, d]`, `material.color`, `position`, optional `mass` (kg; absent / 0 / `immovable: true`  $\rightarrow$  static), optional `castShadow`.
- **cylinder** — dimensions: `[radiusTop, radiusBottom, height, radialSegments]`, `material.color`, `position`, optional `mass`, optional `immovable`.
- **model** — `filename` (absolute URL to an OBJ), `position`, optional `scale` (uniform), optional `orientation: [rx, ry, rz]` (radians, XYZ Euler), optional `color`, `castShadow`, `immovable`.
- **fossbot** — robot spawn marker. `position` + `orientation`. Exactly one per stage.

Common fields: `name` (debug string), `position: [x, y, z]` in meters Y-up, `orientation` in radians.

1. **Loader** — `sim-v2/src/stages/loader.ts` exports `loadStage(name: string, scene, world): Promise<StageHandle>`. Fetches `/stages/${name}.json`, parses, dispatches each entry to a builder.
2. **Builders** — one per object type, each returns `{ visuals: Object3D[], colliders: Collider[] }`:
  - `buildFloor` — `THREE.PlaneGeometry` rotated to lie on  $Y=0$ ; large static cuboid( $w/2$ ,  $0.005$ ,  $h/2$ ) collider replacing the Phase 3 stub ground; loads texture via `THREE.TextureLoader` with `repeat/offset` if present.
  - `buildCube` / `buildCylinder` — Three primitive + Rapier cuboid / cylinder collider; dynamic body if `mass > 0` and not `immovable`, else static (`fixed`).
  - `buildModel` — `OBJLoader` (already in `three/examples/jsm`); for `immovable` items use `RAPIER.ColliderDesc.trimesh(vertices, indices)` from the loaded geometry; for dynamic ones fall back to a single oriented bounding box (good enough for v2's needs — full convex decomp is the kind of thing CoACD would give us, deferred).
  - `buildFossbot` — sets the chassis body's translation/rotation to the spawn pose; does NOT recreate the body.
3. **Color parsing** — accept both 'orange' and decimal numbers (65280). One helper `parseColor(value): THREE.ColorRepresentation`.
4. **Asset paths** — JSON references absolute URLs like `/js-simulator/models/static/Ramp.obj` and `/js-simulator/textures/white_rect.png`; `sim-v2/public/js-simulator/` owns those files locally so the simulator no longer depends on `front-end/public`.
  - Start with (a). Fall back to (b) only if Vite proxying gets in the way.
5. **Stage handle** — `StageHandle` exposes `dispose()` that removes meshes from the scene and removes static colliders from the world (track them for clean teardown). Required for the lil-gui stage switcher in the next phase.
6. **Stage registry** — `sim-v2/src/stages/index.ts` enumerates known stages by listing the names already shipped under `public/stages/`. Used by the GUI dropdown in Phase 6.

**Verify:** - `stage_white_rect`, `stage_ramp`, `stage_object` all load without error and look visually equivalent to V1. - Robot spawns at the position the JSON says, not the hardcoded Phase 3 spawn. - Static cubes/cylinders/ramps are immovable; dynamic ones (`mass > 0`) get pushed when the robot bumps them. - Switching from one stage to another via a temporary keyboard shortcut leaves no leaked colliders (check with `world.bodies.len()` / collider count).

### 1.6.5 Phase 5 — Custom raycast wheel force model (replaces vehicle controller)

Goal: replace the static cylinder wheel colliders with a per-wheel raycast force model that gives real suspension, real traction, and clean ramp behavior on a 2-wheel differential-drive chassis. Caster stays a passive `Ball` collider; only the two drive wheels go through the model.

**Why not `DynamicRayCastVehicleController`?** It was the initial design (and the entire previous draft of this phase). Dropped during ramp testing because:

- The controller is car-shaped: it assumes `N` steered/driven raycast wheels. v2 is **two-wheeled with fixed (non-steering, non-rotating) wheels** — most of the controller’s machinery (steering angle, axle Cs, wheel rotation tracking) is dead weight or actively fights the differential-drive model.
- The controller writes chassis velocities directly each `updateVehicle()`, which fights any external torque or impulse we apply (tried this for in-place rotation in an earlier session — see “Option 1 torque” history; the controller’s lateral-friction term anchored rotation to the wheel-axle line, not the chassis CoM).
- On inclines the velocity-write made it hard to add a slope-hold term without arguing with the controller.

The custom model applies forces (not velocities) at each wheel contact point, leaving the chassis as a normal Rapier dynamic body integrated by `world.step()`.

**Pre-step — drop the wheel cylinder colliders.** The original `ROBOT_COLLIDERS` left/right cylinders are still filtered out of the chassis build (via `createRobotBody({ skipDriveWheels: true })`). Two solid colliders riding the ground while we also raycast = contact thrash.

**Per-frame model (in `physics/vehicle.ts`).** For each wheel `i` `{LEFT, RIGHT}`:

1. Compute the wheel attachment point in world space (chassis-local connection point transformed by chassis pose).
2. Cast a ray downward (chassis-local `(0,-1,0)` rotated to world) from the attachment, length `suspensionRestLength + maxSuspensionTravel + wheelRadius`.
3. If the ray hits the floor:
  - **Suspension force:** spring along the suspension axis, magnitude `k * compression - c * compressionVelocity`. Applied at the wheel contact point on the chassis body.
  - **Longitudinal force** (engine / brake): `motorForce * input[i]` along chassis-forward at the contact point. `input[i]` `[-1, 1]` from `setDrive(left, right)`. Brake is applied as opposing force when input is zero or sign-flipped.
  - **Side-friction force:** opposite to the lateral component of the contact-point velocity, scaled by `sideFriction`. Keeps the chassis from sliding sideways while turning.
  - **Slope-hold:** an extra anti-rollback force `chassis.mass() * g * sin(slope) * slopeFactor` along chassis-forward, scaled to the wheel’s contact normal. Lets the robot brake-hold on a steep ramp without engine input.
4. If the ray misses, the wheel is airborne — no force applied this frame.

**Tunable settings (live in the GUI; defaults in `createDefaultVehicleSettings(wheelRadius)`).**

- `motorForce` = 12 (Newtons at full throttle, per wheel).
- `brakeStrength` — opposing force when input is zero / brake is held.
- `suspensionStiffness`, `suspensionDamping`, `suspensionRestLength`, `maxSuspensionTravel`.
- `sideFriction` — side-friction stiffness for in-place stability.
- `slopeFactor` — slope-hold gain (0 disables).
- `wheelRadius` is sourced from the loaded wheel mesh, not hardcoded.

**API.**

```

createVehicle(world, chassis): VehicleHandle
VehicleHandle.setDrive(left: number, right: number) // each in -1..1
VehicleHandle.update(dt) // call before world.step()
VehicleHandle.telemetry: VehicleTelemetry // per-wheel contact, susp, slip, force

```

setDrive only wakes the chassis when input is non-zero (lets the body sleep when idle, fixing the earlier idle-drift).

Differential drive mapping (in App.tsx keyboard handler).

- W/S → both wheels  $\pm 1$ .
- A/D → opposite signs for in-place rotation, or asymmetric magnitudes when combined with W/S for skid-steer.
- Space → idle/brake (setDrive(0, 0) and chassis sleeps when velocity decays).

**Visual wheel sync.** Wheels are non-steering (the chassis is two-wheeled differential-drive — no Ackermann, no yaw-axis rotation on the wheels themselves), but they do **spin visually** around their body-local X axle and bob with the suspension:

- Roll: `wheelSpin[i] += (longitudinalContactVelocity[i] / wheelRadius) * dt` while the wheel is touching ground; falls back to `input * freeSpinSpeed * dt` when airborne so the wheel still spins under throttle. `visualWheels[i].rotation.x = wheelSpin[i]`.
- Suspension bob: `visualWheels[i].position.y = baseY + (suspensionRestLength - currentSuspensionLength)` so the mesh dips into the fender as the spring compresses.

The wheel meshes stay parented under `robot.root` (the chassis frame) — only the local `position.y` and `rotation.x` are touched per frame.

**Verify:** - WASD drives the robot. - Forward/reverse symmetric. - A/D rotates roughly in place around the chassis center; D = clockwise from above. - No drift on flat ground when no input. - Robot brake-holds on a 30° ramp (slope-hold factor active). - Suspension visibly works (telemetry reports compression on bumps).

### 1.6.6 Phase 5b — UI scaffolding (parallel to Phase 5)

Goal: make the simulator usable for hand-tuning during ramp work. These pieces all landed in the May 5–6 ramp-testing burst and support Phase 5/Phase 8 iteration. They are visual / UX scaffolding, not physics — kept in their own phase so Phase 7 (Debug menu) stays focused on lil-gui parameter binding.

1. **Camera modes** (`src/ui/cameraControls.ts`, `src/ui/cameraTypes.ts`). A view-mode button next to the Debug GUI cycles **Orbit** → **Follow** → **Top**:
  - Orbit: the existing OrbitControls camera; pose is preserved when leaving and restored when returning.
  - Follow: third-person chase camera tracking behind the robot.
  - Top: fixed eagle-eye for surveying the whole stage.
2. **Splash screen** (`src/ui/splashScreen.ts`). Startup overlay reporting load progress (world init, stage load, robot load, vehicle ready). Visibility persisted in the World folder of the Debug GUI.
3. **Movement presets** (`src/ui/movementPresets.ts`). Physics-driven keyboard shortcuts for canned moves (e.g. spawn, drive forward N seconds, pivot 90°). Used to repro tuning scenarios reliably.
4. **text stage entry type** (`buildText` in `stages/builders.ts`). Canvas-backed sprite label, optional `onFloor` flag for ground-aligned labels. Used in `stage_ramp.json` to call out which lane is which fixture.
5. **“Remember last stage” toggle** (`src/debug/...stageFolder + localStorage`). When on, the last selected stage is restored on reload.
6. **Reset-to-spawn** action exposed in the Stage folder of the Debug GUI; same path used after a stage swap.
7. **Runtime controls** wired in `App.tsx` and the World folder: pause/step/time-scale, telemetry overlay visibility, stage collider wireframe visibility (preserved across stage swaps).

**Verify:** - Switching camera modes is instant; orbit pose round-trips when toggling away and back. - Splash screen disappears once everything is ready; reappears on hard reload. - Stage swap → reset-to-

spawn → drive: clean, no leaked state. - “Remember last stage” survives a page reload. - Pause + step-once advances physics by exactly one fixed step.

### 1.6.7 Phase 6 — Stage object collider visualization

Goal: extend the existing “Show Colliders” toggle (which already wireframes the robot’s primitive colliders) to also wireframe stage object colliders, so when something looks off in physics it’s obvious whether the visual or the collider is wrong.

1. **Builders return collider metadata.** Each stage builder already returns `{ object, collider }`. Add a sibling debug wireframe (matching the collider’s shape and transform) and a small marker so the loader can group them.
2. **Stage builds a hidden colliders group.** `loader.ts` collects every debug wireframe into a single `THREE.Group` named `stage_colliders`, parented to the scene root, `visible = false` by default.
3. **Toggle wires through the existing GUI.** Reuse the Debug GUI’s “Show Colliders” boolean — flipping it toggles both the robot colliders group AND the active stage’s colliders group.
4. **Trimesh models:** bake the same vertices/indices used for the Rapier `trimesh` collider into a `THREE.LineSegments` (`EdgesGeometry` over a `BufferGeometry`) so wireframes match the physics exactly, including baked transforms.
5. **Stage swap cleanup.** `StageHandle.dispose()` removes the colliders group along with the visual objects.

**Verify:** - Toggling “Show Colliders” with `stage_object` shows magenta cube/cylinder wireframes co-located with the visual cubes/cylinders. - Toggling on `stage_ramp` shows the trimesh wireframe of `Ramp.obj` aligning with the visible mesh. - Toggling off restores the clean view. - Stage swap leaves no orphan wireframes in the scene.

### 1.6.8 Phase 7 — Debug menu (lil-gui) and reload affordances

1. Install `lil-gui`. Mount one panel, top-left, collapsible.
2. Folders + bindings:
  - **World:** gravity, time scale, paused, physics wireframes (`world.debugRender`), step-once button.
  - **Robot:** mass, body linear/angular damping, reset-to-spawn button, log-state button. Manual movement along axes.
  - **Wheels** (per side or shared): radius, suspension stiffness, max travel, friction slip, side friction, compression, relaxation, engine force max, brake force max.
  - **Drive:** forward speed cap, turn force scale, deadzone.
  - **Stage:** dropdown populated from the Phase 4 stage registry (e.g. `stage_white_rect`, `stage_ramp`, `stage_object`, plus the new ramp set added in Phase 7). Selecting calls the loader’s `dispose()` on the current stage and `loadStage()` on the new one.
3. **Console output panel:** small DOM overlay (not `lil-gui`) printing every N frames: position, velocity, yaw, per-wheel suspension length, per-wheel ground contact, per-wheel slip. A “log now” button dumps the full state to `console.log` for copy/paste.

**Verify:** every parameter changes behavior live (no rebuild needed except mass/radius which need rigid-body recreation). Stage reload swaps geometry without leaking colliders. Logs are useful, not spammy.

### 1.6.9 Phase 8 — Ramps and behavior validation

1. **One combined stage,** `sim-v2/public/stages/stage_ramp.json`, with side-by-side fixtures so we can tune by driving between lanes without reloading. Each fixture is either a tilted `cube` collider (oriented cube support added in Phase 4) or a `model` referencing a ramp OBJ. Floor labels via the new `text` entry type call out which lane is which.
2. Fixtures in the combined stage:
  - 15° short ramp (1 m).
  - 15° long ramp (4 m) — used as an up-flat-down bridge lane.
  - 30° steep ramp (2 m).
  - Mixed bump lane: short + long + a small bump for suspension travel.

3. Tune the custom raycast model (Phase 5) until:
  - **Climb**: robot climbs the 15° short and long ramps at full throttle without stalling.
  - **Hold**: with throttle released, the slope-hold term keeps the robot stationary on the 30° ramp.
  - **Slip**: with very low `sideFriction`, robot visibly slides sideways on a side-tilted incline — sanity check.
  - **Descent**: robot can drive down the 30° ramp controllably with brake input; no flipping.
  - **Edge drop**: robot driving off the side of a ramp drops cleanly without sticking.

**Verify**: record success of each behavior in `PROGRESS.md` (one entry per tuning session is fine). Floor labels mean we can screenshot a single stage to document everything.

#### 1.6.10 Phase 9 — Replace `sim-dev/`

Only after Phase 8 lands cleanly:

1. `git mv sim-dev sim-dev-old` (or delete after confirming).
2. `git mv sim-v2 sim-dev`.
3. Update any root-level scripts/links that pointed at `sim-dev`.
4. `PROGRESS.md` entry.

### 1.7 Critical files to mirror values from (read-only references, not imports)

- `sim-dev/src/robot/robotV2.ts` — `PART_DEFAULTS`, `PART_ADJUST`, `PART_COLOR`, `WHEEL_DEFAULTS`, fender-center math, axis-correction pivot. Phase 2 reimplements this from scratch but copies the *numbers*.
- `sim-dev/src/physics/world.ts` — Rapiere init pattern (`RAPIER.init()` then `new RAPIER.World(gravity)`).
- `sim-dev/src/physics/robotBody.ts` — body-local visual offset rotated by body quaternion on sync. Phase 3 reuses this idea.
- `sim-v2/public/js-simulator/models/robots/v2/*.stl` and `*_coacd.stl` — local assets served by the v2 app.

### 1.8 Verification summary (end-to-end)

After all phases, the new tree should pass these by hand:

1. Page loads, robot is upright and centered on a flat plane, gizmo shows correct orientation.
2. Drop test (toggle physics on): robot settles cleanly on its two wheels + caster, no penetration, no jitter.
3. WASD drives: forward, reverse, skid-steer turn — all symmetric and predictable.
4. Suspension visibly works (wheel meshes compress over bumps).
5. Climbs short and long 15° ramps under full throttle.
6. Holds on 30° ramp with brake.
7. Slip parameter visibly affects traction.
8. Edge drop off ramp doesn't get stuck.
9. Stage reload from GUI works without leaking colliders.
10. Restart-the-robot button respawns at origin without leftover state.

### 1.9 Tooling / workflow notes (not phase work)

- **Post-commit hash hook** (`.github/hooks/post-commit`, May 6). After every commit, the hook replaces the active `PENDING` placeholder in `PROGRESS.md` with the real commit short hash, stages the update, and creates a follow-up chore: `progress commit hash commit`. Per-worktree opt-in; documented in `AGENTS.md`.
- **`AGENTS.md` replaces `CLAUDE.md`** (May 6). Repository operating rules now live in `AGENTS.md` (commit behavior, progress-log handling, hook automation, simulator-source boundaries). The old `CLAUDE.md` was deleted as superseded.

### 1.10 Decisions (confirmed)

- Directory: `sim-v2/`.

- Wheel mesh: copy `wheel.obj` + `wheel.mtl` into `sim-v2/public/wheel/`.
- Caster: free `Ball` collider on chassis (option A).
- Debug GUI: `lil-gui`.

### 1.11 Beyond the plan

Features and utilities built that were not listed in the original phase scope:

- `positionPresets` / `positionStore` — save/restore robot poses per stage; persisted to `localStorage`.
- `collidersTunerFolder` — per-collider live tuning (size, position, rotation, friction, density, restitution) with dump-to-clipboard and reset-to-defaults.
- `visualTunerFolder` — fender and wheel position tuning with CAD-mm units; mirror toggle; dump emits paste-ready constants for `v2.ts`.
- `checkSuspensionHealth` (`util/suspension.ts`) — static sanity check for suspension stiffness vs robot weight; logs critical/soft/ok verdict.
- `Telemetry overlay copy` — `copyRobotState` dumps full state to clipboard from the Telemetry GUI folder.

### 1.12 Still to confirm during Phase 3 tuning (resolved)

- ~~Robot mass target~~ — settled at **2.0 kg** with inertia `{x: 0.014, y: 0.007, z: 0.009}`.